

Question 1

Explain the fundamental differences between DDL, DML, and DQL commands in SQL. Provide one example for each type of command.

Answer

SQL commands are categorized based on their functionality in managing and manipulating data in a database. The three important categories are **DDL, DML, and DQL**.

1. DDL (Data Definition Language)

DDL commands are used to **define and modify the structure of database objects** such as tables, schemas, and indexes. These commands affect the **structure of the database rather than the data stored in it**.

Common DDL Commands

- CREATE
- ALTER
- DROP
- TRUNCATE

Example

```
CREATE TABLE Students (  
    student_id INT,  
    name VARCHAR(50),  
    age INT  
);
```

This command creates a table named **Students** with three columns: student_id, name, and age

2. DML (Data Manipulation Language)

DML commands are used to **insert, update, and delete data stored in database tables**. These commands deal with **actual data manipulation**.

Common DML Commands

```
INSERT  
UPDATE  
DELETE
```

Example

```
INSERT INTO Students (student_id, name, age)
VALUES (1, 'Rahul', 20);
```

3. DQL (Data Query Language)

DQL commands are used to **retrieve data from a database**. The main command used in DQL is **SELECT**.

Example

```
SELECT * FROM Students;
```

Question 2

What is the purpose of SQL constraints? Name and describe three common types of constraints, providing a simple scenario where each would be useful.

Answer

SQL constraints are rules applied to table columns to **ensure the accuracy, reliability, and integrity of the data stored in a database**. They prevent invalid data from being entered into the database tables.

1. PRIMARY KEY

A **Primary Key** uniquely identifies each record in a table. It **cannot contain NULL values** and must always be **unique**.

Scenario:

In a student database, every student must have a unique **Student ID** so that each record can be identified easily.

Example

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

2. NOT NULL

The **NOT NULL constraint** ensures that a column cannot contain **NULL or empty values**.

Scenario:

A student's name should always be present in the database; therefore, the **name column cannot be left empty**.

Example

```
CREATE TABLE Students (
```

```
student_id INT,  
name VARCHAR(50) NOT NULL  
);
```

3. UNIQUE

The **UNIQUE constraint** ensures that all values in a column are **different from each other**.

Scenario:

Each student must have a unique **email address** so that no two students share the same email.

Example

```
CREATE TABLE Students (  
  
    student_id INT,  
  
    email VARCHAR(100) UNIQUE  
  
);
```

Question 3

Explain the difference between LIMIT and OFFSET clauses in SQL. How would you use them together to retrieve the third page of results, assuming each page has 10 records?

Answer

The **LIMIT** and **OFFSET** clauses are used in **SQL** to control the number of rows returned by a query.

LIMIT

The **LIMIT** clause specifies the maximum number of records that should be returned by a query.

Example

```
SELECT * FROM Students  
  
LIMIT 10;
```

This query returns only the first 10 records from the table.

OFFSET

The **OFFSET clause** specifies the number of rows to **skip before starting to return rows**.

Example

```
SELECT * FROM Students  
LIMIT 10 OFFSET 10;
```

This query skips the **first 10 records** and then returns the **next 10 records**.

Retrieving the Third Page of Results

If each page contains **10 records**, then:

- Page 1 → OFFSET 0
- Page 2 → OFFSET 10
- Page 3 → OFFSET 20

SQL Query

```
SELECT * FROM Students  
LIMIT 10 OFFSET 20;
```

Explanation:

This query skips the first **20 records** and returns the **next 10 records**, which represent the **third page of results**.

Question 4

What is a Common Table Expression (CTE) in SQL, and what are its main benefits? Provide a simple SQL example demonstrating its usage.

Answer

A **Common Table Expression (CTE)** is a temporary result set that can be referenced within a SQL query. It is defined using the **WITH clause** and is commonly used to simplify complex queries.

Benefits of CTE

- Improves readability of SQL queries
- Makes complex queries easier to understand
- Helps organize large SQL statements

- Can be referenced multiple times within the same query

Example

```
WITH HighSalaryEmployees AS (  
    SELECT name, salary  
    FROM Employees  
    WHERE salary > 50000  
)
```

```
SELECT * FROM HighSalaryEmployees;
```

Explanation:

In this example, the CTE named **HighSalaryEmployees** stores employees whose salary is greater than **50,000**, and the main query retrieves those records.

Question 5

Describe the concept of SQL Normalization and its primary goals. Briefly explain the first three normal forms (1NF, 2NF, 3NF).

Answer

Normalization is the process of organizing data in a database to **reduce redundancy and improve data integrity**. It divides large tables into smaller related tables and establishes relationships between them.

Goals of Normalization

- Reduce data redundancy
- Improve data consistency
- Eliminate data anomalies
- Improve database organization

First Normal Form (1NF)

A table is in **First Normal Form** if:

- Each column contains **atomic values**
- There are **no repeating groups**
- Each record is unique

Example (Not in 1NF)

Student	Subjects
Anjali	Math, Science

Correct Form (1NF)

Student	Subject
Anjali	Math
Anjali	Science

Second Normal Form (2NF)

A table is in **Second Normal Form** if:

1. It is already in **1NF**
2. All non-key attributes are **fully dependent on the entire primary key**

This removes **partial dependency**.

Third Normal Form (3NF)

A table is in **Third Normal Form** if:

1. It is already in **2NF**
2. There are **no transitive dependencies**

This means non-key columns should **not depend on other non-key columns**.

/*Question 6 : Create a database named ECommerceDB and perform the following tasks:

1. Create the following tables with appropriate data types and constraints:
 - Categories

- CategoryID (INT, PRIMARY KEY)
 - CategoryName (VARCHAR(50), NOT NULL, UNIQUE)
 - Products
 - ProductID (INT, PRIMARY KEY)
 - ProductName (VARCHAR(100), NOT NULL, UNIQUE)
 - CategoryID (INT, FOREIGN KEY → Categories)
 - Price (DECIMAL(10,2), NOT NULL)
 - StockQuantity (INT)
 - Customers
 - CustomerID (INT, PRIMARY KEY)
 - CustomerName (VARCHAR(100), NOT NULL)
 - Email (VARCHAR(100), UNIQUE)
 - JoinDate (DATE)
 - Orders
 - OrderID (INT, PRIMARY KEY)
 - CustomerID (INT, FOREIGN KEY → Customers)
 - OrderDate (DATE, NOT NULL)
 - TotalAmount (DECIMAL(10,2))
- */

ANSWER

```

create database ECommerceDB ;
use ECommerceDB ;
-- ● Categories
-- ○ CategoryID (INT, PRIMARY KEY)
-- ○ CategoryName (VARCHAR(50), NOT NULL, UNIQUE)
create table Categories(
CategoryID int primary key ,
CategoryName varchar(50) not null
unique);

select * from Categories;
insert into Categories values
(1,"Electronics"),
(2,"Books"),
(3,"Home goods"),
(4,"Apparel");

create table Products(
ProductID INT PRIMARY KEY,

```

```
ProductName VARCHAR(100) NOT NULL UNIQUE,  
CategoryID INT,  
FOREIGN KEY (CategoryID)  
references Categories(CategoryID),  
Price DECIMAL(10,2) NOT NULL,  
StockQuantity INT  
  
);
```

```
INSERT INTO Products VALUES  
(101, 'Laptop Pro', 1, 1200.00, 50),  
(102, 'SQL Handbook', 2, 45.50, 200),  
(103, 'Smart Speaker', 1, 99.99, 150),  
(104, 'Coffee Maker', 3, 75.00, 80),  
(105, 'Novel : The Great SQL', 2, 25.00, 120),  
(106, 'Wireless Earbuds', 1, 150.00, 100),  
(107, 'Blender X', 3, 120.00, 60),  
(108, 'T-Shirt Casual', 4, 20.00, 300);
```

```
select* from Products;
```

```
create table Customers(  
CustomerID INT PRIMARY KEY,  
CustomerName VARCHAR(100) NOT NULL,  
Email VARCHAR(100) UNIQUE,  
JoinDate DATE  
);
```

```
INSERT INTO Customers VALUES  
(1, 'Alice Wonderland', 'alice@example.com', '2023-01-10'),  
(2, 'Bob the Builder', 'bob@example.com', '2022-11-25'),  
(3, 'Charlie Chaplin', 'charlie@example.com', '2023-03-01'),  
(4, 'Diana Prince', 'diana@example.com', '2021-04-26');  
Select* from Customers;
```

```
create table Orders(  
OrderID INT PRIMARY KEY,  
CustomerID INT ,  
FOREIGN KEY (CustomerID)  
references Customers(CustomerID),
```



```
OrderDate DATE NOT NULL,  
TotalAmount DECIMAL(10,2)  
);
```

```
INSERT INTO Orders VALUES  
(1001, 1, '2023-04-26', 1245.50),  
(1002, 2, '2023-10-12', 99.99),  
(1003, 1, '2023-07-01', 145.00),  
(1004, 3, '2023-01-14', 150.00),  
(1005, 2, '2023-09-24', 120.00),  
(1006, 1, '2023-06-19', 20.00);  
Select * from Orders;
```

/*Question 7 : Generate a report showing CustomerName, Email, and the TotalNumberOfOrders for each customer. Include customers who have not placed any orders, in which case their TotalNumberOfOrders should be 0. Order the results by CustomerName.
*/

ANSWER

```
SELECT  
c.CustomerName, c.Email,  
COUNT(o.OrderID) as TotalNumberOfOrders  
from Customers c  
left join Orders o on  
c.CustomerID = o.CustomerID  
group by c.CustomerID, c.CustomerName, c.Email  
ORDER BY c.CustomerName;
```

/*Question 8 : Retrieve Product Information with Category: Write a SQL query to display the ProductName, Price, StockQuantity, and CategoryName for all

products. Order the results by CategoryName and then ProductName alphabetically
*/

ANSWER

```
select
p.ProductName,
p.Price,
p.StockQuantity ,
c.CategoryName
from Products p
join Categories c on
p.CategoryID= c.CategoryID
order by c.CategoryName, p.ProductName;
```

/***Question 9** : Write a SQL query that uses a Common Table Expression (CTE) and a Window Function (specifically ROW_NUMBER() or RANK()) to display the CategoryName, ProductName, and Price for the top 2 most expensive products in each CategoryName.*/

ANSWER

```
WITH RankedProducts AS (
    SELECT
        c.CategoryName,
        p.ProductName,
        p.Price,
        ROW_NUMBER() OVER (
            PARTITION BY c.CategoryName
            ORDER BY p.Price DESC
        ) AS rn
    FROM Products p
    JOIN Categories c
        ON p.CategoryID = c.CategoryID
)

SELECT
    CategoryName,
```

```
    ProductName,  
    Price  
FROM RankedProducts  
WHERE rn <= 2  
ORDER BY CategoryName, Price DESC;
```

USE sakila;

/***Question 10** : You are hired as a data analyst by Sakila Video Rentals, a global movie rental company. The management team is looking to improve decision-making by analyzing existing customer, rental, and inventory data. Using the Sakila database, answer the following business questions to support key strategic initiatives.

Tasks & Questions:

1. Identify the top 5 customers based on the total amount they've spent. Include customer name, email, and total amount spent.
2. Which 3 movie categories have the highest rental counts? Display the category name and number of times movies from that category were rented.
3. Calculate how many films are available at each store and how many of those have never been rented.
4. Show the total revenue per month for the year 2023 to analyze business seasonality.
5. Identify customers who have rented more than 10 times in the last 6 months.

*/

-- **Answer:**

-- 1. Top 5 Customers

```
SELECT c.first_name, c.last_name, c.email, SUM(p.amount) AS total_spent  
FROM customer c  
JOIN payment p ON c.customer_id = p.customer_id  
GROUP BY c.customer_id, c.first_name, c.last_name, c.email  
ORDER BY total_spent DESC  
LIMIT 5;
```

-- 2. Top 3 Categories

```
SELECT cat.name, COUNT(r.rental_id) AS rental_count
FROM category cat
JOIN film_category fc ON cat.category_id = fc.category_id
JOIN inventory i ON fc.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY cat.name
ORDER BY rental_count DESC
LIMIT 3;
```

-- 3. Films per Store

```
SELECT i.store_id,
       COUNT(DISTINCT i.film_id) AS available_films,
       SUM(CASE WHEN r.rental_id IS NULL THEN 1 ELSE 0 END) AS never_rented
FROM inventory i
LEFT JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY i.store_id;
```

-- 4. Monthly Revenue

```
SELECT MONTHNAME(payment_date) AS Month, SUM(amount) AS Total_Revenue
FROM payment
WHERE YEAR(payment_date) = 2023
GROUP BY MONTH(payment_date), MONTHNAME(payment_date)
ORDER BY MONTH(payment_date);
```

-- 5. Frequent Customers

```
SELECT c.first_name, c.last_name, COUNT(r.rental_id) AS rental_count
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
WHERE r.rental_date >= CURRENT_DATE - INTERVAL 6 MONTH
GROUP BY c.customer_id, c.first_name, c.last_name
HAVING COUNT(r.rental_id) > 10;
```